

Dart: objects, null safety and collections

Lecture 2 — Mobile Application Development with Flutter I

Nurkhan Tashimov · Kazakh-British Technical University

WHERE WE LEFT OFF

Last week you ran an app. Today you learn to model things.

1. **Objects**
   Group data and behaviour so the compiler can check your work

2. **Null safety**
   Make the most common crash in mobile apps impossible to write

3. **Collections**
   Transform a list of 200 students into the answer you need, in one line

4. **Dart 3**
   Enums with data, records, and switches the compiler proves complete

Every screen you build for the rest of the course sits on top of today.

HOW TO READ THIS DECK

Two things that repeat on every slide

NEW TERM

Every word you have not met before gets its own slide: the term, one sentence that defines it, a small example, and where you will meet it again.

If a word appears without one of these slides, stop me — that is a bug in my deck.

RED CODE

Red always means exactly one thing: this code does not compile. Not "bad style", not "slow" — the app will not build.

Black-on-teal code is code that works.

✗ THIS IS WHAT RED LOOKS LIKE

You do not need to copy the code down.

The deck goes in the course channel straight after the lecture, and it is the reference for the homework. Listen now, look it up later.

1 YOUR TURN

Same typo in both. Which one wakes you up at 2am — and why?

```
final s = {'name': 'Aida', 'age': 20};
print(s['nmae']);
```

A bag of key–value pairs

```
final s = Student(name: 'Aida', age: 20);
print(s.nmae);
```

An object with a known shape

One of these you find in five seconds. The other one your user finds.

Every answer goes on the board first — including the wrong ones.

THE ANSWER

A class is a contract the compiler checks

✗ RED = COMPILE ERROR

The same typo in both: `nmae` instead of `name`.

print(s['nmae']) → **compiles, runs, prints null.** **print(s.nmae)** → **refused before you save the file.**

Names are checked

A typo in a field name is a compile error, not a null at runtime.

Types are known

s.age is an int. You cannot accidentally add a String to it.

The IDE can help you

Autocomplete, rename, "find usages" — none of that works on a Map key.

Behaviour lives with data

averageGrade belongs next to grades, not in some helper file.

VOCABULARY CHECK

The five words behind that slide

`class`	A blueprint. It says what fields an object has and what it can do. Written once.
`object`	One thing built from that blueprint. Student(name: 'Aida') makes one. Also called an instance.
`field`	A named piece of data stored inside the object — name, age, grades.
`method`	A function that lives inside the class and can use its fields. Called with brackets: s.describe().
`getter`	A value the object computes when you ask for it. Looks like a field, no brackets: s.average.

If any of these five is new to you, say so now. Everything today is built out of them.

ANATOMY

Fields, constructor, getter

```
class Student {
  final String name;        // set once, never changes
  final int age;
  final String? email;      // may legitimately be absent

  Student({required this.name, required this.age});

  double get average => ...;   // computed, not stored

  @override
  String toString() => 'Student($name, $age)';
}
```

final means the field is set once, in the constructor, and never again.

required means the compiler will not let you build a Student without a name.

Both are checked before your app ever runs.

Prefer final everywhere. Make a field mutable only when you can say why.

NEW TERM

final

final means: this field is given a value once, when the object is built, and can never be changed again.

```
class Student {
  final String name;      // set once
  int    absences = 0;    // can change later

  Student({required this.name});
}

final s = Student(name: 'Aida');
s.absences = 3;       // fine
s.name = 'Bob';       // does not compile
```

A field you cannot reassign is a field that cannot surprise you.

Nobody else in the program can change it behind your back, so when you read it you know what is there.

Rule for this course: write final by default. Drop it only when you can say out loud why the value must change.

Where you meet it: *every field of every widget you write from week 3 must be final*

NEW TERM

Named parameters · required

Named parameters are arguments you pass by name instead of by position. required means the compiler refuses to build the object without one.

```
// positional - what is true? what is 3?
Student('Aida', 19, true, 3);

// named - reads itself
Student(name: 'Aida', age: 19,
        isActive: true, course: 3);

Student({required this.name, this.age = 0});
Student(age: 19);   // no name -> refused
```

Curly braces in a constructor mean "these are named".

A named parameter is optional unless you mark it required — and if it is optional, it needs either a default value or a nullable type.

That is the whole rule, and it is the first thing we break in the demo.

Where you meet it: *the entire Flutter API — Container(width: 8, color: Colors.red)*

2 YOUR TURN

Dart has a keyword called `factory` for constructors.

What can a factory do that a normal constructor cannot?

Hint: think about what a constructor is forced to return.

Every answer goes on the board first — including the wrong ones.

THE ANSWER · QUESTION 2

A factory is the one that can decide

A normal constructor MUST return a brand-new object of exactly its own class. **A factory has a body, so it can decide.**

Default

Student({required this.name});
The ordinary case.

Named

Student.guest();
A second door, with a name that explains itself.

const

const Student.unknown();
Known at compile time. Two identical values are ONE object.

factory

factory Student.fromJson(map) { ... }
Has a body — so it can return a cached object, a subclass, or a default.

Each of these four now gets its own slide.

NEW TERM · CONSTRUCTOR 1 OF 3

Named constructor

A second constructor with a name after a dot, for a different way of building the same class.

```
class Student {
  final String name;
  final int age;

  Student({required this.name, required this.age});

  Student.guest() : name = 'Guest', age = 0;
}

final a = Student(name: 'Aida', age: 19);
final b = Student.guest();
```

Dart has no constructor overloading: you cannot write two constructors with the same name and different arguments, the way Java or C++ let you.

Instead you give the second one a name. Student.guest() explains itself; Student('Guest', 0) does not.

The part after the colon is the initialiser list — it sets final fields before the body runs.

Where you meet it: *Colors.black, DateTime.now(), Icon.fromChar — all named constructors*

NEW TERM · CONSTRUCTOR 2 OF 3

const constructor

A constructor whose result is fully known while the code is being compiled — so Dart builds the object once and reuses it everywhere.

```
class Point {
  final int x, y;
  const Point(this.x, this.y);   // all fields final
}

const a = Point(1, 2);
const b = Point(1, 2);
print(identical(a, b));   // true - ONE object

final c = Point(1, 2);    // built at runtime
print(identical(a, c));   // false
```

Two rules and one payoff.

Rule 1: every field must be final, otherwise const is impossible.
Rule 2: the values must be known at compile time.

Payoff: identical const values are literally the same object in memory. Flutter checks that — if the widget is the same object, it skips rebuilding that part of the screen.

That is why const is not decoration.

Where you meet it: *const Text('Hello') — you will type const hundreds of times*

NEW TERM · CONSTRUCTOR 3 OF 3

factory constructor

A constructor with a body, which decides what to return instead of always building a brand-new object.

```
class Student {
  final String name;
  Student(this.name);

  factory Student.fromName(String? raw) {
    if (raw == null || raw.isEmpty) {
      return Student('Unknown');   // decided
    }
    return Student(raw.trim());
  }
}
```

A normal constructor is forced to return a new instance of exactly its own class, and it cannot run code before the fields are set.

A factory can do both: run code first, then return a new object, an object it already had (a cache), or an object of a subclass.

That is the whole difference. Everything else is the same.

Where you meet it: *every model class you write in this course has a factory fromJson*

NEW TERM · A CONVENTION, NOT A KEYWORD

fromJson

A factory constructor that turns the Map a server sends you into a real object of your class.

```
// what arrives from the network:
// {'name': 'Aida', 'age': 19}

factory Student.fromJson(Map<String, dynamic> j) {
  return Student(
    name: j['name'] as String? ?? 'Unknown',
    age:  j['age']  as int?    ?? 0,
  );
}
```

Where you meet it: *week 11, when your app starts talking to a real API*

fromJson is not a Dart keyword. It is a name the whole community agreed on, so every model in every project looks the same.

The map arrives as Map<String, dynamic>: dynamic means the compiler knows nothing about the values. So every single read needs two things —

as int? — the type you expect, which may be missing
?? 0 — what to use when it is missing.

Miss either half and the app crashes on the first user with an incomplete profile.

NEW TERM · A CONVENTION, NOT A KEYWORD

copyWith

A method that returns a new object identical to this one, except for the fields you asked to change.

```
Student copyWith({String? name, int? age}) =>
    Student(
      name: name ?? this.name,
      age:  age  ?? this.age,
    );

final a = Student(name: 'Aida', age: 19);
final b = a.copyWith(age: 20);

// a is still ('Aida', 19)
// b is       ('Aida', 20)
```

Every field is final, so you can never change a Student. What you do instead is build a new one.

Read the pattern: name ?? this.name means "use the new name if I was given one, otherwise keep the old one".

The object you had is untouched, which is exactly what makes state predictable — nobody else holding it sees it change.

Where you meet it: *week 9 — every screen state update in your app is a copyWith*

3 YOUR TURN

Student and Teacher both have a name and an email.

You do not want to write that twice. What are your options?

There are at least four. Collect them all before we judge any of them.

Every answer goes on the board first — including the wrong ones.

THE ANSWER · QUESTION 3

Four options. Three of them are keywords.

`extends`

What am I?

You get the parent's fields AND its code. One parent only. Use when the child really is a kind of the parent.

`implements`

What can I do?

You get the contract and none of the code — you must write every member yourself. Any number of them. This is how test doubles are made.

`with`

What do I borrow?

A reusable piece of behaviour bolted onto a class. Any number. Flutter uses this everywhere, e.g. SingleTickerProviderStateMixin.

4. Composition — no keyword at all

Give the class a Person field instead of inheriting from Person. Boring, and usually the right answer — it is how the whole of Flutter is built.

Default to composition. Reach for extends only when the "is a" sentence is actually true.

NEW TERM · KEYWORD 1 OF 3

extends — inheritance

extends means "this class is a kind of that one". The child gets the parent's fields and its working code, and may replace any method.

```
class Person {
  final String name;
  Person(this.name);
  String describe() => name;
}

class Student extends Person {
  final int course;
  Student(super.name, this.course);

  @override                    // replacing a method
  String describe() => '$name, course $course';
}
```

One parent only. Ever.

@override says "I am deliberately replacing the parent version". It is not required, but leave it out and the compiler cannot catch a typo in the method name.

super.name passes the argument straight up to the parent constructor. super on its own means "the parent version of this".

Where you meet it: *class MyScreen extends StatelessWidget — from next week, every screen.*

NEW TERM · KEYWORD 2 OF 3

implements — an interface

implements means "this class promises the same members as that one" — and gives you none of its code. You write every member yourself.

```
class FakePerson implements Person {
  @override
  final String name = 'Fake';   // even the field

  @override
  String describe() => 'not a real person';
}

// anything expecting a Person accepts this
void show(Person p) => print(p.describe());
show(FakePerson());   // works

// delete an @override -> does not compile
```

In Dart every class automatically defines an interface — the list of its public members. implements says you will provide that list.

The part that surprises everyone: fields count too. name was a field on Person; here it is a member you declare by hand.

Any number of interfaces, unlike extends.

Coming from Java? Dart 3 also has an interface modifier: abstract interface class C {} is a class others may implement but not extend.

Where you meet it: *test doubles, and swapping a real service for a fake one in week 13.*

NEW TERM · KEYWORD 3 OF 3

mixin · with

A mixin is a block of working code you can bolt onto many unrelated classes, without any of them becoming a parent or a child.

```
mixin Greeter on Person {     // only onto a Person
  String greet() => 'Hi, $name';   // may read name
}

class Student extends Person with Greeter { }
class Teacher extends Person with Greeter { }

Student('Aida', 1).greet();   // 'Hi, Aida'
```

Read it as a sentence: Student IS A Person, and it CAN ALSO greet.

on Person is a restriction — this mixin may only be attached to a Person or its subclasses. In exchange, the mixin is allowed to use Person's members, which is how greet() can read name.

A class may have one parent but any number of mixins. If two mixins define the same member, the last one in the with list wins.

Where you meet it: *SingleTickerProviderStateMixin, when you write your first animation*

NEW TERM

abstract class

A class you cannot build directly. It exists to say what its children must have — and to hold the parts they can share.

```
abstract class Person {
  final String name;
  const Person({required this.name});

  String describe();          // no body: children MUST

  String get initials =>      // has a body: free for all
      name[0].toUpperCase();
}

final p = Person(name: 'X');   // cannot build it
class Teacher extends Person { }  // no describe()
```

A method with no body is an obligation. Every child must write one, and the compiler checks.

A method with a body is inherited free.

What you buy: you can hold a List<Person> of mixed students and teachers and call describe() on all of them without knowing which is which. That is polymorphism — one call, many behaviours.

Where you meet it: *StatelessWidget is an abstract class with one abstract method: build*

4 YOUR TURN

What is the single most common runtime crash in mobile apps?

You have all seen it. Some of you saw it this week.

Every answer goes on the board first — including the wrong ones.

THE ANSWER · QUESTION 4 · THE BILLION-DOLLAR MISTAKE

Dart splits every type in two

`String`

Guaranteed to hold text. The compiler proves it can never be null, and optimises accordingly.

`String?`

May hold text, may hold nothing. The compiler forces you to handle the second case before you use it.

These are two different types. You cannot pass a String? where a String is expected — not because of a warning, but because it does not compile. The crash you all just named becomes a class of bug you can no longer write by accident.

"Sound" null safety: the guarantee is strong enough that the compiler optimises on it.

THE TOOLKIT

Five things you will type every day

?.

Call it only if it is not null. The result is nullable.
name?.length

??

Use this value instead, when the left side is null.
name ?? 'Guest'

??=

Assign only if the variable is currently null.
cache ??= load()

!

I promise this is not null. If you are wrong, the app crashes.
name!.length

late — a fifth one: "I will assign this before anybody reads it." Break that promise and you get an error, not a silent null.

NEW TERM

late

late means: the value is NOT optional — I simply cannot give it to you yet. I promise to set it before anybody reads it.

```
late final String token;   // type is String,
                           // NOT String?
void start() {
  token = 'abc123';        // assigned here
}

print(token);   // fine after start()
print(token);   // before start(): throws
// LateInitializationError
```

The mistake everyone makes: thinking late is another way of writing "?". It is the opposite.

String? = the value may honestly be absent, and you must handle that.
late String = the value will definitely be there, just not yet.

Break the promise and you get an error at runtime, not a silent null. That is the trade you are making.

Where you meet it: *controllers created in initState() — week 8, animations*

5 YOUR TURN

name comes from the network and may be null. Which line can crash your app — and when?

```
1   String? name = await fetchName();
2   print(name?.length);
3   print(name ?? 'unknown');
4   print(name!.length);
```

Decide first — then be ready to say why.

Every answer goes on the board first — including the wrong ones.

THE ANSWER

! is a promise, not a conversion

?. and ?? ask the compiler for help. ! tells the compiler to stop helping.

```
print(name!.length);

// If name is null this throws at runtime, on a user's phone,
// in a place the compiler already warned you about.
```

Rule for this course: every ! you write, you must be able to explain. Most of them can be replaced by ?? or an early return.

THE TRAP THAT COSTS HOURS

A local promotes. This field does not.

Works

```
final nick = profile.nickname;
if (nick != null) {
  print(nick.length);  // nick is String here
}
```

Does not compile

```
if (profile.nickname != null) {
  print(profile.nickname.length);
}
// still String? — the field could change
```

nickname is public and not final. It could be reassigned between the check and the use — or, in another library, overridden by a getter that returns something new every call. Dart refuses to guess.

The exact rule (Dart 3.2+): a field promotes only if it is final AND private — _nickname — and you read it inside the same library. Anything else: copy it into a local first.

6 YOUR TURN

You have a list of 200 students, each with a list of grades.

Give me the average grade for the whole group.

Name at least two different ways. One of them fits on a single line.

Every answer goes on the board first — including the wrong ones.

THE ANSWER · QUESTION 6

Both are right. One of them you can read out loud.

What most of you said

```
var total = 0;
var count = 0;
for (final s in group) {
  for (final g in s.grades) {
    total += g;
    count++;
  }
}
final avg = total / count;
```

The one-expression version

```
// expand = flatten a list of lists into one
final all = group
    .expand((s) => s.grades)
    .toList();

// fold = squeeze a list into a single value
final avg = all.fold<int>(0, (a, b) => a + b)
            / all.length;
```

The loop is not wrong. It is just eight lines you have to read before you know what it does.

The next three slides are the three containers, what an Iterable really is, and the handful of methods that make the right-hand version possible.

THREE CONTAINERS

List, Set, Map

`List`

Ordered, duplicates allowed

[90, 85, 90]

The default. Indexed, keeps insertion order.

`Set`

Unordered, no duplicates

{'dart', 'flutter'}

Add the same value twice and nothing happens. Membership checks are fast.

`Map`

Key to value

{'Aida': 84}

Look something up by a key. The keys are unique.

All three are generic: List<Student> is a different type from List<String>, and the compiler knows it.

NEW TERM

Iterable — and why it is lazy

An Iterable is a recipe for a sequence, not the sequence itself. Nothing is computed until somebody actually walks through it.

```
final names = group.map((s) {
  print('working on ${s.name}');
  return s.name;
});            // <- an Iterable

// console so far: nothing at all

final list = names.toList();
// NOW the prints appear
```

A List holds its values. An Iterable holds the instructions for producing them.

map and where both return Iterables. Add .toList(), a for loop, or .length and the work finally runs.

Two consequences: side effects inside map may never happen, and walking the same Iterable twice does the work twice. Both bite people once — today it will be you, in the demo.

Where you meet it: *the demo in fifteen minutes: a print inside map that never prints*

ONE LINE INSTEAD OF A LOOP

map · where · fold

```
final names    = group.map((s) => s.name).toList();
final adults   = group.where((s) => s.isAdult).toList();
final total    = group.fold<double>(0, (sum, s) => sum + s.average);
final best     = group.reduce((a, b) => a.average > b.average ? a : b);
final anyFail  = group.any((s) => s.average < 50);
final allGrades = group.expand((s) => s.grades).toList();

final byName   = {for (final s in group) s.name: s.average};   // a Map
final unique   = group.map((s) => s.name).toSet();             // a Set
```

map and where return a lazy Iterable — nothing runs until you ask for the values.

Forget .toList() and the work simply never happens — you will see exactly that in the demo. · The last two lines are the same idea for the other containers: a for inside { } builds a Map, and .toSet() throws the duplicates away.

BUILDING A LIST IN PLACE

spread · collection-if · collection-for

```
final roster = <String>[
  'Group A',
  ...names,                // spread a whole list in
  if (showGuests) '+ guests',   // only when true
  for (var i = 1; i <= 3; i++)  // generate several
    'free slot $i',
];
```

From week 4 you will write exactly this shape inside a Column:

```
children: [
  Header(),
  if (isLoading) Spinner(),
  for (final s in group) Row(s),
]
```

No helper functions, no builders.

Remember this slide. In three weeks it stops being syntax and becomes how you build screens.

NEW TERM

enum with fields

An enum is a fixed, closed list of named values. Since Dart 2.17 each value can also carry data and the enum can have methods.

```
enum Level {
  beginner('Beginner', 0),    // value + its data
  advanced('Advanced', 85);

  final String label;
  final int minScore;
  const Level(this.label, this.minScore);    // must
                                             // be const
  static Level from(int s) =>
      s >= advanced.minScore ? advanced : beginner;
}

print(Level.advanced.label);    // 'Advanced'
```

Before, an enum was only a list of names and the labels lived in a map somewhere else in the project. Now the data sits with the value.

The constructor must be const, because enum values are compile-time constants — that falls out of what an enum is.

static means the method belongs to the type, not to one value: you call Level.from(90), not advanced.from(90).

Where you meet it: *any fixed set: order status, user role, screen tab*

NEW TERM

record

A record is a small bundle of several values with a type but no name and no methods — for when writing a whole class would be overkill.

```
// positional: two ints, in order
(int, int) minMax(List<int> g) =>
    (g.first, g.last);

final (low, high) = minMax(grades);   // unpacking

// named: the call site reads itself
({String name, int score}) summary(Student s)
    => (name: s.name, score: s.average.round());

final r = summary(aida);
print('${r.name}: ${r.score}');
```

Before records, returning two values meant inventing a class for one use, or returning a List and losing the types.

Positional records are read by position; named ones by name. Once you find yourself counting positions, switch to names.

final (low, high) = … is destructuring: pulling the pieces out into separate variables in one line.

Records are immutable and compare by value.

Where you meet it: *any function that owes you two answers at once*

NEW TERM

sealed class

A sealed class may only be extended inside its own file — so the compiler knows the complete list of its subtypes.

```
sealed class LoadState {}        // this file only

class Loading extends LoadState {}

class Loaded extends LoadState {
  final List<String> items;
  Loaded(this.items);
}

class Failed extends LoadState {
  final String message;
  Failed(this.message);
}
```

Sounds like a restriction, and it is. Look at what you buy with it.

Because nobody can add a fourth subtype from another file, the compiler knows there are exactly three. It can then check that you handled all three — that is the next slide.

Think of it as an enum whose values are allowed to carry different data each.

Where you meet it: *week 9: every screen you build is loading, loaded, or failed*

NEW TERM

pattern · switch expression

A pattern checks the shape of a value and pulls data out of it in one step. A switch expression uses patterns to produce a value instead of running statements.

```
String describe(LoadState s) => switch (s) {
  Loading()              => 'Loading...',
  Loaded(:final items)   => '${items.length} items',
  Failed(:final message) => 'Error: $message',
};        // no default, and none is needed

// delete the Failed line and you get:
// The type 'LoadState' is not exhaustively
// matched by the switch cases.
```

Three new things at once.

1. The switch is an expression: it produces a value, so it uses => and commas, not case and break.

2. Loaded(:final items) is a pattern: it checks the type AND creates a variable holding the field, in one move. No casting.

3. No default. Because LoadState is sealed, the compiler already knows the list is complete — and it refuses to build if you miss one. That is called exhaustiveness.

Where you meet it: *add a fourth state one day, and the compiler walks you to every screen that must handle it*

7 YOUR TURN

An AI wrote this model for us. Name two things wrong with it.

```
class Student {
  String name;
  int age;
  String email;

  Student({this.name, this.age, this.email});
}
```

Every answer goes on the board first — including the wrong ones.

THE ANSWER · QUESTION 7

Four things, and one of them is fatal

```
class Student {
  String name;      // 1. not final
  int age;
  String email;

  Student({this.name, this.age, this.email});
  // 2. non-nullable, not required, no default
  //    -> this does NOT compile
}
// 3. no const constructor
// 4. no toString()
```

1 · Nothing is final

Any code anywhere can change a Student after it is built. And without final there can be no const constructor either.

2 · It does not compile

Non-nullable named parameters with no required and no default are not allowed. I said an AI wrote it — I did not say it works.

3 · No const

So this can never be a cheap widget field. Flutter will rebuild it every single time.

4 · No toString

Every debug print says Instance of 'Student'. You will lose an hour to that at least once.

It took a model nine seconds to write that, and it took you two minutes to find four things wrong with it.

That second skill is the one worth money now — and it is the one this course grades. It is also why AI is not allowed in it: you cannot practise spotting bad code by not writing any.

8 YOUR TURN

Before I run anything — predict.

I have a list of 4 students and I call .map() on it, with a print inside, and I forget .toList().

Commit to a number. Then we watch.

How many lines are printed?

Every answer goes on the board first — including the wrong ones.

SCREENS OFF, WATCH THIS

Live demo

1. The whole app on screen — five sections, one per topic of today
2. Step 1: delete a required, then break const — watch identical() flip
3. Step 2: delete describe() in Teacher — the compiler refuses the class
4. Step 3: the field-promotion error, then the ! crash on a real device
5. Step 4: remove .toList() — the console stays empty. Your prediction?
6. Step 5: delete one case from the switch — "not exhaustive"

Every one of these is an error you will hit in the lab. Today you meet them with me instead of alone at midnight.

THE ANSWER · QUESTION 8

Zero. Nothing was printed at all.

```
final loud = group.map((s) {
  print('processing ${s.name}');   // never runs
  return s.name.toUpperCase();
});

// map returned a recipe. Nobody cooked it.

final list = loud.toList();   // NOW it runs
```

No crash. No warning. No red underline. The analyzer was perfectly happy.

That is what makes laziness worth a slide: it is the one failure today that nothing at all told you about.

If a map has a side effect in it, you wanted a for loop.

Who wrote down zero?

Hands up. Whether you got it or not, you now have a reason to predict before you run — and that is the habit worth keeping from today.

WHAT JUST HAPPENED

Six errors — five of them before the app ran

Caught by the compiler

Missing required · missing describe() · field not promoted · switch not exhaustive

Caught at runtime

Only one: the broken ! promise — and the compiler had already pointed at that line

Silent, no error at all

The forgotten .toList(). Nothing crashed; the work simply never happened.

The lesson

A strong type system moves bugs from your users to your keyboard. That is the whole argument for Dart.

EVERYTHING WE NAMED TODAY

Glossary — objects and hierarchy

`class`	A blueprint: what fields an object has and what it can do	`Name.thing()`	Named constructor — a second way to build the same class
`object`	One thing built from a class. Also called an instance	`const ctor`	Known at compile time; identical values are ONE object
`field`	A named piece of data stored inside an object	`factory`	A constructor with a body that decides what to return
`getter`	A value computed when you read it. No brackets	`fromJson`	Convention: a factory that turns a Map into an object
`final`	Set once when the object is built, never reassigned	`copyWith`	Convention: a new object, identical but for one field
`named param`	An argument passed by name, not by position	`extends`	"Is a kind of." Fields and code inherited. One parent
`required`	The compiler refuses to build the object without it	`@override`	I am deliberately replacing a member of the parent
`constructor`	The code that builds an object of the class	`implements`	Same members promised, none of the code inherited

Every one of these had its own slide. The deck is in the channel; use it while you do the homework.

EVERYTHING WE NAMED TODAY

Glossary — null safety, collections, Dart 3

`mixin / with`	Working code bolted onto many unrelated classes
`abstract`	Cannot be built directly; children must finish it
`polymorphism`	One call, many behaviours, chosen by the real type
`String?`	May hold text or nothing. A different type from String
`?.`	Read it only if it is not null; otherwise the whole thing is null
`??`	Use this value instead, when the left side is null
`??=`	Assign only if the variable is currently null
`!`	A promise it is not null. Wrong -> crash right there
`late`	Not optional — just assigned later than the constructor
`promotion`	The compiler narrows String? to String after your check
`Iterable`	A recipe for a sequence. Lazy: nothing runs until asked
`map/where/fold`	Transform each · keep some · squeeze into one value
`expand`	Flatten a list of lists into one flat sequence
`... (spread)`	Insert a whole collection inside another literal
`enum`	A closed list of named values, which may carry data
`record`	Several values with a type, no class and no methods
`sealed`	Closed list of subtypes, so the compiler knows them all
`pattern`	Check the shape and pull the data out in one step
`exhaustive`	Every case handled — and the compiler proves it

LAST FIVE MINUTES

Kahoot

8 questions on everything we did today

Not an exam, not part of your grade. It exists so that both of us find out right now what actually stuck — while I can still do something about it.

If more than half the room misses a question, that topic opens the next lecture.

Phones out. Everything else away.

BEFORE THE PRACTICE SESSION

Homework

1 **Model a library**

Author, a Genre enum with labels, Book and Library. Final fields, a named constructor, a factory fromJson, a getter. Pure Dart — dart run, no UI.

2 **Make null impossible**

One book in the data has no pages and no genre. Handle it with ?. and ?? — zero ! in the whole folder, and dart analyze must be clean.

3 **One expression each**

Six queries with map / where / fold, a record from statsOf, and a sealed ShelfState described by a switch expression with no default.

Full brief and the exact data set are in the course channel. Push before the practice session — that is your ticket in.

Next lecture: the widget tree, and why everything in Flutter is a widget.